

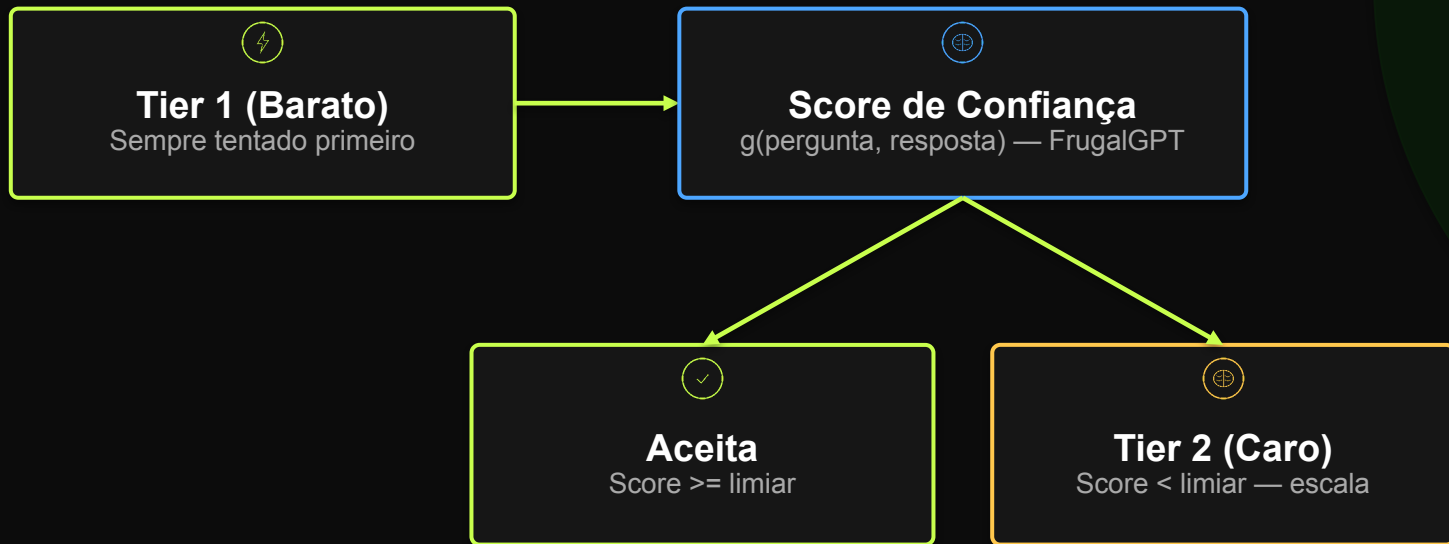
Arquitetura

Enterprise (PT-4)

Arquitetura de Sistemas com IA · Módulo 5.4

Prof. Ahirton Lopes, Ph.D.

Cascata, Não Binário



FrugalGPT (Chen, Zaharia, Zou — Stanford, TMLR 2024): cascata dos mais baratos aos mais caros; economia de até 98% mantendo a performance do melhor modelo individual.

No próprio paper, um exemplo de calibração: aceita o modelo mais barato com score acima de 0,96, tenta um intermediário acima de 0,37, só então escala pro mais caro.

Já em Produção, **Em Muitas Camadas**

Azure AI Foundry Model Router

- 01** Roteia entre várias famílias e tamanhos de modelo — de nano a frontier, incluindo Claude e Llama — com três modos: Balanceado (aceita 1-2% de diferença de qualidade pro mais barato), Custo (5-6%), e Qualidade (ignora custo).

Mesmo a AWS, sofisticada, começa em dois

- 02** O Amazon Bedrock Intelligent Prompt Routing, sistema real e bem documentado, com economia de custo publicada em torno de 30% em uso geral, hoje exige exatamente dois modelos da mesma família — prova de que cascata de dois tiers já entrega ganho real antes de qualquer sistema evoluir pra três ou mais.

O princípio é sempre o mesmo, em qualquer número de tiers

- 03** Modelo mais barato primeiro, escalação só quando um sinal de confiança justifica — a mesma lógica do FrugalGPT, seja em dois tiers locais ou em seis camadas de um provedor cloud.

TrialForge: O Gateway Final

```
async function processarComCascata(pergunta, estudoId) {  
  if (!verificarOrcamento(estudoId, custoMaximoPossivel)) return bloqueado;  
  const { similaridade: confianca, clausula } = await buscarClausula(embedding);  
  if (intencao === 'sintese_csr') {  
    rascunho = await gerarComTier(MODELO_TIER2, pergunta, clausula); // regra fixa  
  } else {  
    rascunho = await gerarComTier(MODELO_TIER1, pergunta, clausula); // tenta barato  
    if (confianca < LIMIAR_CASCATA) {  
      rascunho = await gerarComTier(MODELO_TIER2, pergunta, clausula); // escala  
    }  
  }  
  registrarGasto(estudoId, custoDoTierUsado);  
  registrarAuditoria({ estudoId, tierUsado, escalouCascata, gastoAcumulado });  
}
```

Roda de graça na sua máquina — dois tiers reais via Ollama; um terceiro tier pago entraria no mesmo ponto.

Missão Prática #05

01

Calibre sua própria cascata

Escolha dois modelos (ou dois tiers, se usar API paga) e calibre o limiar de score com dado real do seu contexto — não copie o limiar do TrialForge.

02

Implemente orçamento por tenant

Adicione controle de orçamento que bloqueia ANTES da chamada ao modelo, não depois — reaproveite a lição do Módulo 5.2 no seu próprio protótipo.

03

Rode e documente os três comportamentos

Execute o protótipo e registre: um caso que o Tier 1 resolve sozinho, um caso que escala pro Tier 2, e um caso bloqueado por orçamento.

Entrega: cascata calibrada com evidência + orçamento por tenant + log dos 3 comportamentos + reflexão de crescimento, num único arquivo.

Cinco Módulos. Uma Arquitetura Só.

Começamos decidindo quando um componente merece ser não-determinístico, no Módulo 1. Passamos pela anatomia de um agente único, pela coordenação entre vários, pelos padrões que tornam cada resposta mais precisa, mais barata e mais rápida, e fechamos hoje em como isso tudo roda, é observado, é implantado e tem seu custo controlado numa plataforma real, com dezenas de estudos ao mesmo tempo.

O TrialForge começou como um agente respondendo uma pergunta. Termina como a Vitalis Platform — a mesma arquitetura, multiplicada, com cada decisão que fizemos ao longo do caminho ainda visível dentro dela.

Regra Fixa Não É Resolvido.

A síntese de CSR sempre vai pro modelo caro, desde o Módulo 1.3 — regra fixa, porque errar ali custa demais. Mas regra fixa não é resolvido: e se existisse um modelo pequeno, treinado só nisso, barato e preciso o bastante pra nunca mais precisar escalar?

Essa é a pergunta que abre a próxima disciplina — Processamento de Dados e Fine-Tuning de Modelos: quando vale treinar seu próprio modelo, como preparar um dataset de verdade, fine-tuning via API da OpenAI, LoRA pra não gastar fortuna, e como avaliar se o modelo treinado ficou bom mesmo. Fine-tuning entra como mais uma peça na arquitetura, não a substitui.